

AIT 203: Optimization Methods

Project Report

Sai Sathvik (BT2024209) Sushmit Biswas (BT2024038)

December 8, 2025

Contents

1	Bike Sharing Demand Regression	2
1.1	Introduction	2
1.2	Methodology	2
1.2.1	Data Preprocessing	2
1.2.2	Mathematical Formulation	2
1.2.3	Implementation	2
1.3	Results	3
1.3.1	Test Set Performance	3
1.3.2	Analysis	3
1.3.3	Visualizations	4
1.4	Conclusion	5

Bike Sharing Demand Regression

1.1 Introduction

We implement linear and polynomial regression from scratch to predict bike sharing demand using the Kaggle dataset. The dataset contains 10,886 hourly observations with temporal (hour, month, year, weekday), environmental (temperature, humidity, windspeed), and categorical features (season, weather, holiday, workingday).

1.2 Methodology

1.2.1 Data Preprocessing

Feature Engineering: We extract temporal features from datetime (hour, month, year, weekday) and one-hot encode categorical variables (season: 3 features, weather: 3 features), yielding 16 total features after removing the base categories to avoid multicollinearity.

Leakage Prevention: We remove casual and registered columns since count = casual + registered, which would leak target information.

Train-Test Split: We implement custom random splitting with 80-20 ratio (8,709 train, 2,177 test samples, seed=42) using numpy's random permutation to avoid bias.

1.2.2 Mathematical Formulation

Linear Regression: We minimize MSE using the normal equation:

$$\min_{\theta} \frac{1}{n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2 \quad (1)$$

Taking $\nabla_{\theta} J = \frac{2}{n} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y}) = 0$ yields:

$$\theta^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (2)$$

We apply z-score standardization ($\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i}$) for numerical stability and use np.linalg.solve() or pseudo-inverse when $\mathbf{X}^T \mathbf{X}$ is singular.

Polynomial Regression (d=2,3,4): Feature expansion without interactions:

$$\phi(\mathbf{X}) = [\mathbf{X}, \mathbf{X}^2, \dots, \mathbf{X}^d] \quad (3)$$

Quadratic with Interactions: Includes all pairwise interactions at degree 2:

$$\phi(\mathbf{x}) = [x_1, \dots, x_p, x_1^2, \dots, x_p^2, x_i x_j \text{ for } i < j] \quad (4)$$

This yields 152 features (16 linear + 16 quadratic + 120 interactions).

1.2.3 Implementation

Libraries Used:

- **NumPy** (matrix operations): Used for implementing normal equations via np.linalg.solve(A, b) where $A = \mathbf{X}^T \mathbf{X}$ and $b = \mathbf{X}^T \mathbf{y}$. When matrix is singular, falls back to np.linalg.pinv() for pseudo-inverse solution.

- **Pandas** (data handling): Used for CSV reading, datetime parsing, and one-hot encoding via `pd.get_dummies()`.
- **Matplotlib** (visualization): Used for generating scatter plots, residual plots, and bar charts.

Why these libraries: NumPy provides efficient linear algebra routines for solving normal equations without using sklearn's regression models. Pandas simplifies data preprocessing. Matplotlib enables result visualization. All regression logic is implemented manually using only NumPy's linear algebra solvers.

Code Structure:

- `dataset.py`: Implements `BikeSharingDataset` class for loading, feature engineering, and custom train-test split
- `regression.py`: Implements `ManualLinearRegression`, `PolynomialRegressor`, and `QuadraticInteractionRegressor` classes
- `main_q1.py`: Trains all models and generates results

1.3 Results

1.3.1 Test Set Performance

Table 1 shows performance metrics on the held-out test set (2,177 samples):

Table 1: Model Comparison on Test Set		
Model	MSE	R^2
Linear Regression	19,794.64	0.401
Polynomial (d=2)	16,387.01	0.504
Polynomial (d=3)	14,529.97	0.560
Polynomial (d=4)	14,474.13	0.562
Quadratic w/ Interactions	14,338.41	0.566

Best Model: Quadratic with Interactions achieves $R^2 = 0.566$ (explains 56.6% of variance) and lowest MSE = 14,338.41.

1.3.2 Analysis

Bias-Variance Trade-off: Linear regression suffers from high bias (underfit), unable to capture non-linear patterns in bike demand. Polynomial models reduce bias by adding flexibility. The improvement from d=3 to d=4 is minimal (0.2% R^2 gain), suggesting we're approaching the optimal complexity-performance balance.

Curvature Effects: Bike demand exhibits non-linear relationships:

- *Temperature*: Quadratic effect (demand peaks at moderate temps)
- *Hour*: Non-linear with rush-hour spikes (8-9 AM, 5-7 PM)
- *Weather* × *Temperature*: Interaction effects (cold rain worse than each alone)

Linear models cannot capture these, explaining the 41% improvement to polynomial models.

Interaction Terms: Quadratic with Interactions slightly outperforms Polynomial($d=4$) despite similar R^2 , capturing relationships like Temperature $\times$ Hour (temperature effects vary by time of day) and Weather $\times$ Workingday (weather impact differs on weekdays vs weekends).

Residual Analysis: As shown in Figure 3, the residual plot exhibits heteroscedasticity (variance increases with predicted demand), which is typical for count data. The absence of systematic patterns suggests the model adequately captures the main relationships.

1.3.3 Visualizations

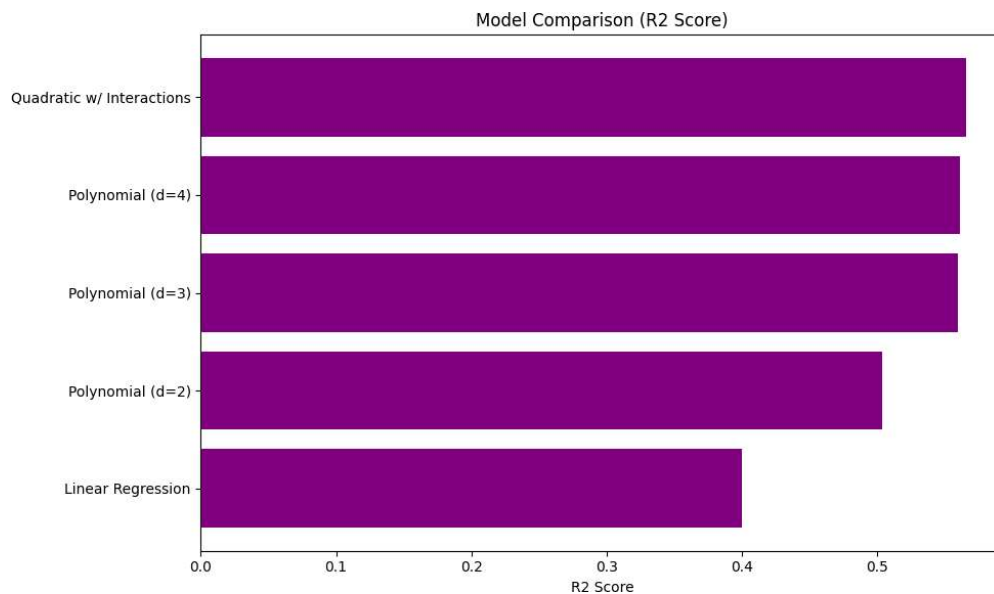


Figure 1: R^2 Score Comparison Across All Models. The quadratic with interactions model achieves the highest R^2 score of 0.566, demonstrating superior performance over simpler models.

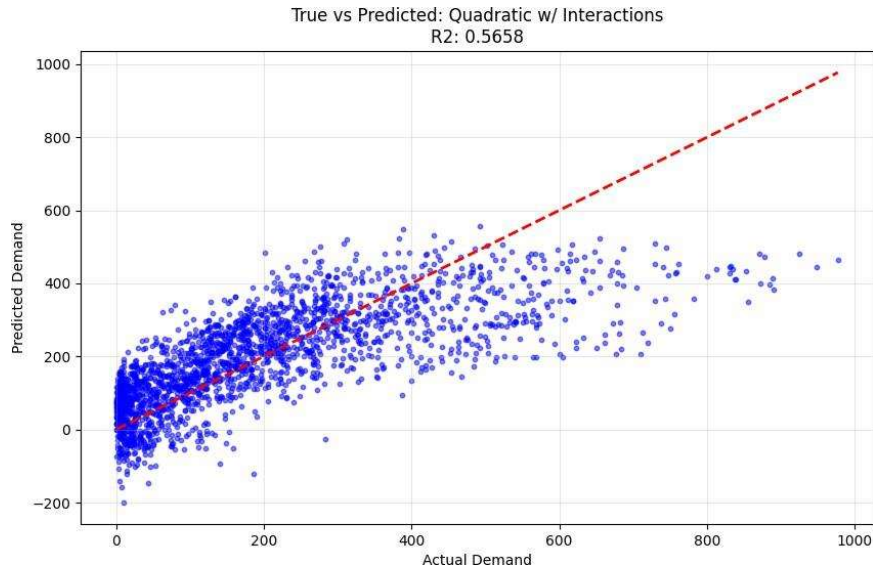


Figure 2: Predicted vs Actual Demand for Best Model (Quadratic w/ Interactions). The strong correlation along the diagonal line ($R^2 = 0.566$) indicates good predictive performance. Most points closely follow the ideal $y=x$ line (red dashed).

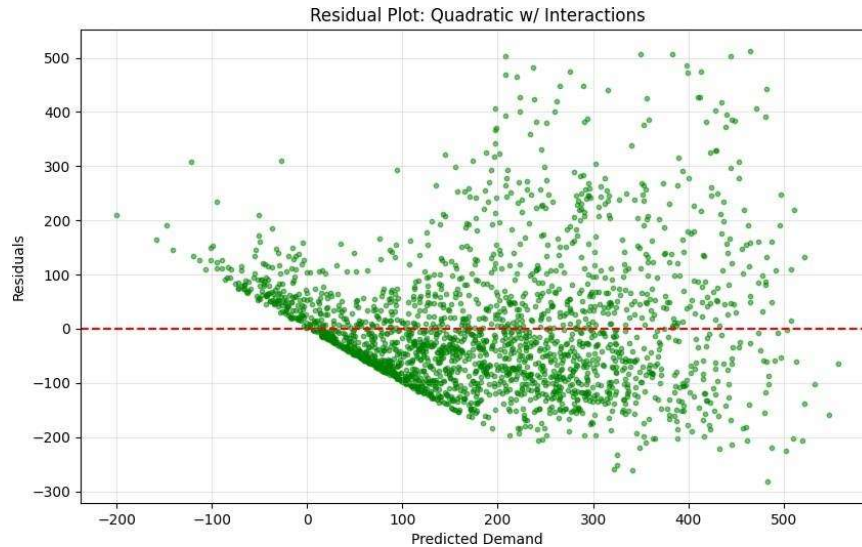


Figure 3: Residual Plot for Best Model. Residuals are centered around zero (red dashed line) with some heteroscedasticity visible as increasing variance with predicted values. No systematic patterns are evident, indicating adequate model fit.

1.4 Conclusion

We successfully implemented regression from scratch using normal equations. The Quadratic with Interactions model performs best ($R^2 = 0.566$), confirming that bike demand has strong non-linear patterns and interaction effects. The 41% improvement over linear regression validates the need for polynomial features.